

Gold Dice RL — cuánto vale saber la respuesta

Trabajo Práctico 1 — Aprendizaje por Refuerzos — Inteligencia Artificial y Neurociencias

1. Antes de entrenar: cuánto hay para ganar

Casi todos los informes de este TP van a decir algo como “*nuestro agente sacó X, el baseline sacó Y*”. El problema es que ese par de números no dice si X es bueno. ¿Faltaban 10 puntos o 300?

Así que empezamos por otro lado: **resolvimos el juego exacto**.

La clave es que **points** no aparece en ninguna transición de `env.py`. No afecta la tirada, ni los costos, ni la tormenta: solo se acumula. Entonces el valor de un estado no depende de los puntos, y de las nueve variables de la observación quedan cuatro que importan —turno, oro, dados, bonus— más los escudos. `roll_sum` ya está sumado al oro cuando toca decidir, y `stored_value` se pliega en la transición.

Eso deja un problema chico. Con inducción hacia atrás desde el turno 30 (donde la respuesta es trivial: el oro se evapora, así que se puntúa todo), y con la distribución conjunta exacta de (suma, máximo) de n dados calculada por convolución, sale el número:

El juego vale 642.45 puntos esperados jugado perfecto.

Verificado de dos maneras: la programación dinámica predice 642.45, y hacer jugar esa política 2.000 partidas simuladas da 645.7 ± 2.9 . Coinciden.

A este solver lo llamamos **el oráculo**. No es un agente de aprendizaje y no se entrega al torneo: es la vara. Sirve para tres cosas que sin él son imposibles: un denominador honesto, saber *en qué turnos* falla el agente aprendido, y separar habilidad de suerte.

Y reencuadra el problema de entrada: **SimpleExpectancy**, el baseline “**inteligente**” de la cátedra, **captura el 53 % del juego. Deja casi la mitad sobre la mesa.**

2. Estado, acciones, recompensa

Acciones. El ambiente ofrece **SCORE** paramétrica: con 400 de oro hay 401 acciones **SCORE** distintas. Nuestra hipótesis inicial fue que colapsarla era el error caro del enfoque tabular. La medimos volviendo a resolver el juego con el espacio mutilado a propósito. Con k libre el óptimo vale 642.45; **dejando solo SCORE(oro) vale exactamente lo mismo**. Costo cero, porque el juego óptimo puntúa una sola vez —en el turno 30— y puntúa todo. El control negativo confirma que la medición discrimina: prohibir “puntuar todo” y dejar solo la mitad baja el óptimo a 641.33. Trabajamos entonces con seis acciones discretas, y esa reducción está medida, no supuesta.

Estado. Lo habitual es aprender “qué tan buena es esta jugada en esta situación”. Nosotros aprendemos otra cosa: **qué tan bueno es el lugar donde quedás después de jugar**. El motivo es que en este juego lo que hace tu acción es totalmente predecible —pagar 42 de oro y sumar un dado es aritmética, no tiene azar— y el azar recién aparece después, con la tormenta y la tirada siguiente. Cuando el problema tiene esa forma conviene aprender el estado resultante, que en la literatura se llama *afterstate* (Sutton & Barto §6.8; es la representación con la que se hizo TD-Gammon). La ganancia es concreta: muchas situaciones distintas terminan en el mismo lugar y comparten aprendizaje, así que hacen falta muchas menos partidas.

El estado que aprendemos es (turno, oro, dados, bonus, escudos). El oro llega hasta 800 y no puede tener una casilla por valor, pero tampoco se puede agrupar de a diez: los precios del juego son 18, 26, 34, 42... , y si 41 y 42 caen en la misma casilla **el agente no distingue “puedo comprar el dado” de “no puedo”**. Usamos entonces una grilla fina abajo (de a 1 hasta 96, donde están todos los precios) y cada vez más gruesa arriba, interpolando entre puntos.

Recompensa. La nativa, sin modificar: los puntos al puntuar. Lo que sí cambiamos es la **parametrización del valor**:

$$V(f) = \Phi(f) + R(f), \quad \Phi = \text{oro} + \text{guardado} + 5 \text{ escudos} + \text{turnos restantes} \cdot \text{dados} \cdot (3.5 + \text{bonus})$$

y la tabla guarda R , no V : calculamos a mano la parte fácil y que la tabla aprenda solo la corrección. Φ es pura contabilidad: la plata que tenés, los activos a lo que costaron, y lo que van a producir tus dados si no comprás nada más.

El motivo es de señal contra ruido. Una partida puede terminar en 300 o en 900 —la desviación es de unos 130 puntos— y las decisiones de los primeros turnos valen 1 o 2. Aprender V directo es tratar de medir 1 punto adentro de 130 de ruido, y no converge en ningún presupuesto razonable. Φ se lleva la parte grande y predecible, y sobre lo que queda las mismas partidas rinden muchísimo más. Cuánto importa: **sin esto el agente saca 105 puntos; con esto, 527**. La técnica se llama *potential-based reward shaping* (Ng, Harada & Russell, 1999) y tiene un teorema detrás que garantiza que no cambia cuál es la política óptima: Φ mueve desde dónde arranca la búsqueda, no adónde llega.

3. Algoritmo e hiperparámetros

Es Q-Learning, escrito sobre el estado resultante en vez de sobre el par (situación, jugada). Mantiene lo esencial: el agente aprende cuál habría sido la mejor jugada aunque en ese momento haya probado otra, que es lo que se llama aprendizaje *off-policy*.

Le agregamos una segunda tabla, y el motivo fue un síntoma raro: la versión de una sola tabla **empeoraba cuanto más entrenaba** —81.0 % del óptimo a 250 mil partidas, 80.0 % a 500 mil, 79.5 % a 750 mil. No es ruido: baja siempre.

La causa es que el agente se pregunta “¿cuál es mi mejor jugada?” y se queda con el valor de esa mejor. Pero sus valores son estimaciones con error, y **quedarse siempre con el máximo de varios números que tienen error**

da un resultado más alto que la realidad. El error se concentra justo donde menos datos hay, así que el agente termina persiguiendo lugares que parecen buenos solo porque están mal medidos, y con más entrenamiento se refuerza. Se llama sesgo de maximización, y la solución de manual es **Double learning** (van Hasselt, 2010): dos tablas, una elige cuál es la mejor jugada y la otra dice cuánto vale. Como sus errores son independientes, el sesgo se cancela.

Hiperparámetro	Valor	Por qué
γ	1	Horizonte finito, fijo y conocido: no hay nada que descontar. Con $\gamma = 0.995$ el pago del turno 30 se multiplica por 0.865, en un juego cuyo único cobro grande es ese.
α	$\max(0.002, (1+v)^{-0.5}) \times 0.015$	Cada casilla lleva su propio contador v de visitas: las que vio mil veces se mueven poco, las nuevas se mueven mucho. Un α único obliga a elegir entre aprender rápido lo frecuente y aprender bien lo raro.
ε	$0.15 \rightarrow 0.02$	Lineal en el 60 % del entrenamiento. Barrido de 11 configuraciones; más exploración empeora.
λ	0	Probamos trazas de elegibilidad ($\lambda = 0.3, 0.5, 0.9$). Todas empeoraron.
episodios	hasta 3×10^6	Mejor checkpoint según la banda de desarrollo; el máximo cae cerca de 4×10^5 (§6).

Semillas: el entrenamiento usa un rango disjunto de las tres bandas de evaluación.

4. Resultados

$n = 20.000$ episodios, intervalo de confianza al 95 %, banda de desarrollo.

Agente	Media	IC 95 %	% óptimo
Oráculo (DP exacta, <i>no compite</i>)	644.10	[642, 646]	100.3 %
Nuestro agente	527.06	[525, 529]	82.0 %
Solo el potencial Φ (cero aprendizaje)	490.41	[488, 493]	76.3 %
Q tabular clásico, rangos amplios + SCORE_ALL	364.21	[363.7, 364.7]	56.7 %
SimpleExpectancy (cátedra)	343.40	[342, 345]	53.5 %
Q tabular clásico, topes chicos, $\gamma = 0.995$	314.58	[314.1, 315.1]	49.0 %
Q tabular clásico, topes chicos, $\gamma = 1$	292.63	[292.0, 293.2]	45.5 %
TD sobre afterstates sin Φ	105.06	—	16.4 %
RandomLegal (cátedra)	63.03	[62.5, 63.6]	9.8 %

Sin sobreajuste. Todas las decisiones se tomaron mirando una sola banda de semillas. Medido en las tres, el agente final da 527.06 (desarrollo, donde se decidió todo), 526.27 (control, mirada una sola vez) y 525.42 (pública, la del enunciado). La diferencia entre desarrollo y control es +0.80 con error estándar 1.37: ruido.

Por qué 20.000 y no las 1000 del enunciado: con $\sigma \approx 100$, mil partidas dan un margen de ± 6.3 puntos, o sea que **una diferencia de 4 puntos entre dos agentes no es una diferencia**. Y acá no se puede usar el truco habitual de comparar sobre el mismo azar, porque en `env.py` el mismo generador sirve los dados y las tormentas y la cantidad de dados depende de la política (§7). La única defensa contra el ruido es el tamaño de muestra.

5. Ablaciones: qué costó cada decisión

Mismo presupuesto (400 mil episodios), cambiando una cosa por vez.

Cambio	De	A	Δ
Topes $\text{dados} \leq 5$, $\text{bonus} \leq 4$, $\text{oro} \leq 240 \rightarrow$ rangos que cubren el juego óptimo	292.6	354.8	+62.2
SCORE de {TODO, MITAD} $\rightarrow$ {TODO}	354.8	364.2	+9.4
γ de 0.995 $\rightarrow$ 1, con rangos amplios	354.6	354.8	+0.2
γ de 0.995 $\rightarrow$ 1, con topes chicos	314.6	292.6	-21.9
Tabla Q discretizada $\rightarrow$ afterstates + Φ + Double	364.2	527.1	+162.9
Potencial Φ apagado $\rightarrow$ encendido	105.1	527.1	+422.0

Los topes cuestan 62 puntos. El juego óptimo llega a 9 dados, bonus 8 y 797 de oro; el 38 % de sus decisiones ocurren con más de 5 dados. Con esos topes, todo el endgame —donde se decide el 70 % del puntaje— cae en una sola celda de la tabla. El agente no aprende mal: no puede ver.

$\gamma < 1$ **solo ayuda cuando la representación está rota**, y es la fila más interesante. Con rangos amplios, $\gamma = 0.995$ y $\gamma = 1$ son indistinguibles; con topes chicos, $\gamma = 0.995$ es 22 puntos **mejor**. Si la representación no distingue el endgame, la política correcta es inexpresable y ser miope es lo mejor disponible —y el descuento hace miope al agente. O sea que $\gamma < 1$ estaba **tapando una falla de representación**, no resolviendo un problema de horizonte. Quien mire solo el número concluye “ $\gamma = 0.995$ anda mejor” y se lleva la lección al revés.

Y la descomposición incómoda, que reportamos porque es verdad: de los 527 puntos, **490 vienen de cómo escribimos el problema y 37 del aprendizaje**. Sin el control “ Φ solo” le habríamos atribuido al algoritmo un mérito que es del modelado.

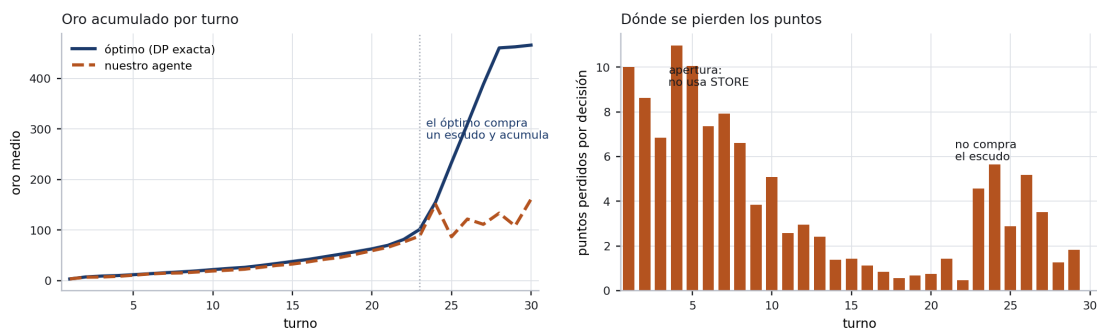
6. Qué no funcionó, y por qué

Faltan 116 puntos hasta el óptimo, y con el oráculo se puede ver exactamente dónde. Para cada jugada calculamos cuánto valor destruyó: lo que valía la situación antes, menos lo que vale después de jugar. Da cero si jugaste óptimo y nunca da negativo. Es lo que se llama **arrepentimiento**: $V^*(s) - [r + \mathbb{E} V^*(s')]$.

El error dominante es no comprar el escudo del turno 23: el agente termina con 0.01 escudos de media contra 0.73 del óptimo, y por eso acumula 183 de oro final contra 484. Es un **óptimo local de política**: el escudo solo rinde si después acumulás, y acumular solo conviene si tenés escudo. Salir de ahí exige cambiar dos decisiones a la vez, y la exploración estándar prueba una cosa por vez: compra el escudo, ve que con su forma de jugar no le sirve, y lo descarta. Probamos tres esquemas pensados para eso — ε por episodio estilo Ape-X, ε -z-greedy con exploración temporalmente extendida (Dabney et al., 2020), y ε alto sostenido— y **ninguno lo rompió**.

Pero la causa de fondo se ve mirando los valores. En ese estado el óptimo separa la mejor acción de la peor por **13.6 puntos sobre un valor de 600: un 2.3 %**, y el error de estimación del agente es de 18 a 29 —más grande que la diferencia entera que tiene que detectar. **El agente no razona mal: su función de valor es correcta al 4 % y la decisión pide 2 %**. Con $\sigma \approx 130$, resolver 13.6 puntos pide del orden de 1.400 muestras independientes por par de estados y el agente tiene ~ 250 por peso: falta un factor de seis.

Eso es una predicción falsable, así que la probamos de las dos maneras. **Una falló**. Subir los episodios de 1.5 a 5 millones no mejoró nada —el agente se estanca en 81 % y después baja—; achicar la grilla de 173 a 114 nodos sí ayudó, pero +3.1 puntos, no el salto que predecía la cuenta. El cuello no es *cuántas* muestras hay sino **de dónde vienen**: los estados que importan —turno 24, mucho oro, un escudo puesto— se visitan solo cuando la exploración compra el escudo, y al turno siguiente la política greedy vuelve a no acumular. Triplicar los episodios triplica unas muestras que nunca contienen la continuación correcta. Es *exploración profunda*, y la cuenta de tamaño de muestra lo pasaba por alto porque supone muestras representativas.



Izquierda: el agente construye un motor casi igual al óptimo pero no acumula detrás de un escudo, y cobra temprano. Derecha: el arrepentimiento no está repartido, se concentra en la apertura y en el turno del escudo.

7. Casos borde del ambiente

Cuatro cosas que están en `env.py` y no en el enunciado, y que cambian la estrategia.

La tormenta cae después de tu acción, así que los puntos son inmunes y solo se toca el oro que decidiste no puntuar: el oro es efectivo arriba de la mesa, los puntos son plata en el banco, y el ladrón pasa después de tu movida. Corolario medido: gastar un turno cuesta $0.075 \times \text{oro}$ (22.76 puntos con 300 de oro) — y **con un escudo, 0.04**.

El turno 30 no tiene tirada después, así que todo el oro sin puntuar se evapora y comprar cualquier cosa ahí es tirar oro. No lo hardcodeamos: en la formulación por afterstates el valor de cualquier estado del turno 30 es 0, y el `argmax` elige `SCORE(todo)` solo.

STORE_BEST_DIE clona el dado, no lo guarda: la suma de la tirada ya se cobró, y la acción paga 4 para volver a cobrar el dado más alto el turno siguiente. Es la acción más usada del juego óptimo. Y **las semillas no son comparables entre agentes** (§4), el caso borde más caro si se ignora, porque invalida cualquier comparación fina. Ninguna de las piezas anticipa por separado la estrategia óptima: construir barato con `STORE_BEST_DIE` hasta el turno 13, comprar dados hasta el 22, **un** escudo en el 23 —uno solo, porque en 7 turnos se espera ~ 1.05 tormentas— y acumular detrás de ese muro hasta cobrar 463 de oro de una sola vez en el turno 30.

8. Qué sigue y qué aprendimos

Qué sigue. El experimento de §6 dice dónde apretar y dónde no: más partidas no sirve, hay que cubrir estados que la política actual nunca visita. Dos caminos: exploración profunda con funciones de valor aleatorizadas (Osband et al.), que produce desviaciones sostenidas de varios turnos en vez de una sola; y una red en lugar de una tabla, para que el valor del estado con escudo se aprenda en parte desde estados parecidos sin tener que visitarlo.

Qué aprendimos. Tres cosas, en orden de cuánto nos sorprendieron.

Primero: **resolver el juego antes de entrenarlo cambió todo el trabajo**. No porque el oráculo diera la respuesta —no se entrega—, sino porque convirtió cada opinión en una medición. Nuestra hipótesis inicial sobre `SCORE` era falsa y lo supimos en diez minutos en vez de en tres días.

Segundo: **el techo lo puso la representación, no el algoritmo**. 490 de 527 puntos vienen de cómo está escrito el estado. Y el caso de γ muestra el corolario feo: un hiperparámetro puede “funcionar mejor” justamente porque compensa un error estructural, y el número solo no lo delata.

Tercero, y es lo que menos esperábamos: **el agente no fallaba por no entender el juego, sino por no tener precisión suficiente**. Estaba a 28 puntos de la respuesta correcta sobre un valor de 600; donde las acciones difieren en 2 %, aprender es un problema de estimación antes que de razonamiento. De ahí salió la lección de método: hicimos dos predicciones cuantitativas sobre cómo cerrar esa brecha y **una falló**. Que fuera falsable es lo que la hizo útil —su fracaso identificó la causa real, qué estados se visitan y no cuántas veces, mucho más rápido de lo que lo habría hecho seguir probando hiperparámetros.